AWS Policy & Governance

Terraform Project — Interview Q&A

Real Scenario Based Questions with Production-Grade Answers

Quick Reference — Topics Covered

Interview Questions & Answers

Q1: You deployed an IAM policy to enforce MFA before S3 delete — but S3 objects were still getting deleted without MFA. What went wrong and how did you fix it?

This was a three-layer problem I debugged systematically.

First problem — Policy was never attached to the user. In IAM, creating a policy and enforcing it are two separate things. The policy existed in AWS but had no effect because it was never linked to any identity.

Second problem — I was attaching policies directly to individual users. This doesn't scale. If a new developer joins tomorrow, someone has to manually attach every policy. The fix was to create an IAM Group, attach all policies to the group, and add users to the group. Now any new user automatically inherits all security policies just by being added to the group.

Third problem — The condition used BoolIfExists with MultiFactorAuthPresent = false. This only evaluates when the MFA key exists in the session. With a standard username/password login, that key is absent entirely — so the condition skips evaluation and the action gets allowed. The fix was

adding a second statement with Bool (not BoolIfExists), so both cases are covered: key present but false, and key missing entirely.

⋆ Bonus Point for Interviewer

In production, I would go further and implement SCPs — Service Control Policies at the AWS Organization level.
SCPs apply to every user in every account automatically, with no manual attachment needed.
Even an admin cannot bypass an SCP, making it the strongest enforcement layer in AWS.

Q2: What is AWS Config and how does it differ from CloudWatch? When would you use each?

They solve fundamentally different problems.

AWS Config answers: 'Are my resources compliant with my rules?' It tracks configuration state and changes — who changed what, when, and whether the result violates a defined policy. It's compliance-focused.

CloudWatch answers: 'Is my application healthy right now?' It tracks operational metrics — CPU usage, error rates, latency, custom application metrics. It's observability-focused.

Concrete example: An EC2 instance loses its encryption tag.
→ CloudWatch: notices nothing, because the app is still running fine
→ AWS Config: immediately flags it as NON_COMPLIANT against the required-tags rule

When to use each:
→ CloudWatch — monitoring uptime, alerting on CPU spikes, tracking request errors
→ AWS Config — detecting when someone opens a security group to 0.0.0.0/0, or disables S3 encryption
→ Both together — Config catches the compliance violation, CloudWatch + SNS sends the alert to the team

⋆ Bonus Point for Interviewer

A mature setup connects both: Config detects a rule violation → triggers a CloudWatch Event → SNS sends a Slack or email alert.
This gives you real-time compliance alerting without anyone manually checking the Config dashboard.

Q3: Explain the IAM policy evaluation logic. If a user has both an Allow and a Deny for the same action, what happens?

AWS uses an explicit deny-wins model. The evaluation order is:

Step 1 — Explicit Deny check: If any policy explicitly denies the action, it is blocked immediately. Full stop.
Step 2 — Explicit Allow check: If there is an explicit allow and no deny, the action is permitted.
Step 3 — Implicit Deny: If nothing explicitly allows it, the action is denied by default.

So in the case of Allow + Deny on the same action — Deny always wins, no exceptions.

This matters a lot in practice. In this project, the MFA delete policy has Effect: Deny. Even if someone has AdministratorAccess (which allows everything), the explicit Deny from the MFA policy overrides it. The user cannot delete S3 objects without MFA, regardless of their other permissions.

The one exception is the AWS root account — root bypasses IAM policies. That's why enabling MFA on root is a separate Config rule in this project.

⋆ Bonus Point for Interviewer

SCPs add another layer: they restrict what IAM policies can even allow.
Order with SCPs: SCP → Resource Policy → IAM Permission Boundary → IAM Identity Policy.
An IAM Admin cannot grant permissions that the SCP doesn't permit — this is why SCPs are the gold standard for enterprise control.

Q4: In your Terraform code you used `depends_on` in the bucket policy. Why is that necessary and what happens without it?

Terraform builds a dependency graph and provisions resources in parallel by default for speed. Without explicit guidance, it may try to create the S3 bucket policy at the same time as the public access block.

The problem: if the bucket policy is applied before the public access block is fully in place, it can result in a conflict or race condition — AWS may reject the policy because the bucket's access settings are in a transitional state.

depends_on = [aws_s3_bucket_public_access_block.config_bucket_public_access_block] tells Terraform: "Do not create the bucket policy until the public access block resource is fully provisioned and confirmed."

I also caught a bug in the original code where the resource name in depends_on was missing the _block suffix — it referenced a resource that didn't exist. This would have caused a terraform apply failure with a 'reference to undeclared resource' error.

The general rule: use depends_on when a resource needs another to be in a specific state, but there is no implicit reference between them that Terraform can auto-detect.

⋆ Bonus Point for Interviewer

Terraform infers most dependencies automatically when you reference one resource inside another.
depends_on is only needed for hidden dependencies — cases where the relationship is operational, not syntactic.

Q5: Your project has 7 AWS Config rules. How does Config evaluate them — on every API call, or on a schedule?

AWS Config rules use two trigger types, and you can use both on the same rule.

Configuration change trigger — evaluates a rule whenever a relevant resource is created, modified, or deleted. For example, the S3 encryption rule fires every time any S3 bucket configuration changes. This gives near real-time detection.

Periodic trigger — evaluates on a fixed schedule: every 1, 3, 6, 12, or 24 hours. This is used for rules that check account-level state rather than individual resource changes. ROOT_ACCOUNT_MFA_ENABLED is a good example — there's no configuration change event for enabling MFA on root, so it needs to be checked periodically.

In this project, the managed AWS rules like S3_BUCKET_PUBLIC_WRITE_PROHIBITED use change-based triggers, while ROOT_ACCOUNT_MFA_ENABLED and IAM_PASSWORD_POLICY use periodic evaluation.

The recorder must be enabled and the delivery channel must be configured before any rules can evaluate — that's why the depends_on chain exists in config.tf: recorder → delivery channel → recorder status → rules.

⋆ Bonus Point for Interviewer

You can also trigger a manual evaluation with: aws configservice start-config-rules-evaluation --config-rule-names rule-name
This is useful after deploying a fix — you can verify compliance immediately without waiting for the next scheduled evaluation.

Q6: What is the difference between an IAM Policy attached to a user vs attached to a Group vs an SCP? Which did you use and why?

Three different layers, each with different scope and strength.

User-level attachment: Policy applies only to that specific user. If you have 50 developers, you attach 50 times. When someone joins, you manually attach. When someone leaves, you manually detach. Any missed attachment is a security gap. This was the original flaw in this project.

Group-level attachment: Policy is attached once to the group. Users inherit all group policies automatically by being members. A new developer joins → add them to the group → they get all policies instantly. A developer leaves → remove from group → all policies revoked. This is what I implemented as the fix.

SCP — Service Control Policy: Applied at the AWS Organization or Organizational Unit level. Applies to every account and every user within that boundary automatically. Cannot be overridden even by an account's root user. This is the enterprise-grade solution.

For this project I used Group-level because it's a single-account demo. In a real company with multiple AWS accounts across teams, I would layer both — SCPs at the org level for non-negotiable rules, and Groups within each account for role-based access control.

⋆ Bonus Point for Interviewer

SCPs are guardrails, not grants. They restrict the maximum permissions available, but don't grant anything by themselves.
A common pattern: SCP denies all regions except us-east-1 and eu-west-1 at the org level, then IAM policies grant specific service access within those regions.

Q7: Your S3 bucket has both server-side encryption and a bucket policy that denies insecure transport. Aren't these redundant? Why have both?

They protect against completely different threats — this is the defense-in-depth principle.

Server-side encryption (AES256) — protects data at rest. If AWS's physical storage media were somehow accessed or if a disk were removed from a data center, the data is unreadable without the decryption key. It's storage-level protection.

Deny insecure transport (DenyInsecureTransport policy) — protects data in transit. It blocks any request that arrives over HTTP instead of HTTPS. This prevents man-in-the-middle attacks where someone intercepts network traffic between a client and S3.

A scenario where you'd need both: imagine an attacker on the same network intercepts an HTTP upload request. Without the transit policy, they capture the data mid-flight in plain text. Without encryption, even if they somehow accessed the stored object directly, they'd read it in plain text.

With both in place, data is protected at every stage of its lifecycle — during transmission and during storage. Removing either one creates a gap in the security posture.

⋆ Bonus Point for Interviewer

For production, upgrade from AES256 to aws:kms encryption.
KMS gives you: full audit trail of every decryption via CloudTrail, automatic key rotation, and ability to instantly revoke access by disabling the key.
AES256 is AWS-managed and has no audit trail — you can't see who decrypted what.

Q8: If terraform destroy is run on this project, what happens to the compliance data stored in S3? Is that acceptable?

The S3 bucket has force_destroy = true, which means terraform destroy will delete the bucket and all its contents — including every Config snapshot and history file stored there. All compliance audit data is permanently lost.

Whether that's acceptable depends entirely on the context.

For a demo or learning project like this one — yes, it's fine. You want clean teardown without manually emptying the bucket first.

For a production governance account — absolutely not acceptable. Compliance data often has legal retention requirements. SOC 2, HIPAA, PCI-DSS all require audit trails to be retained for 1–7 years depending on the framework. Deleting them could be a compliance violation itself.

The production fix has two parts:

→ Remove force_destroy = true from the bucket resource

→ Add an S3 lifecycle policy to transition old data to Glacier after 90 days (much cheaper) and set a retention period before deletion

Additionally, the bucket should have Object Lock enabled in compliance mode — this makes objects legally immutable for a defined retention period, even an admin cannot delete them before the period expires.

⋆ Bonus Point for Interviewer

S3 Object Lock in Compliance mode is the strongest protection — not even the root user can delete locked objects before expiry.

Governance mode allows deletion by privileged users, while Compliance mode allows no exceptions — use the right one based on your regulatory requirements.

Tip: Always close with 'In production I would...' — it shows you think beyond the task.